Homework: Optimization for Machine Learning

Instructions

Delivery: provide a commented jupyter notebook

This is an individual homework, but feel free to discuss with other students about your results. Use `PyTorch` for the implementations. You can use built-in optimizers if they are available (especially for part 2). You can use large language models (LLMs, like chatgpt) for writing and coding (very convenient for data generation, defining objective functions and plots), but you must verify their outputs. Do not rely on them blindly. Discuss your choices, how experiments went and difficulties encountered. See below for more detailed evaluation criteria.

1 Speed restart

1. Implement Nesterov accelerated gradient method with speed restart and Nesterov accelerated gradient method with vanishing friction (without restart), take $b = 3$.
2. Compare both algorithms on the Rosenbrock function in dimension 2.
3. Which algorithm is the best? Provide your own explanation according to intuitions or results seen in class.

2 A scaling law for the stepsize of gradient method

The goal of this part is to provide recommendations for choosing the stepsize when training a model, as the size increases. Here's the learning task; a code is provided on moodle to generate data and train the model.

Data generation. We generate n couples of points (t_i, y_i) of the form

$$y_i = \sin(t_i) + \epsilon_i,$$

where $t_i \in [-T, T]$ are equally spaced and $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$.

- Suggested values: $n = 200$, $T = 5$, $\sigma = 0.1$.
- You may change these values if needed (avoid trivial ones such as $\sigma = 0$, $n = 1$, etc.).

Model We train a small neural network $h(x, \cdot)$ with one hidden layer of size d:

$$h(w_1, w_2, b_1, b_2, t) = w_2 \,\mathrm{ReLU}(w_1 t + b_1) + b_2,$$

where $w_1, b_1 \in \mathbb{R}^d$, $w_2, b_2 \in \mathbb{R}$ and $\mathrm{ReLU}(z) = \max(0, z)$ is applied componentwise. On the whole data set, the learning problem is then

$$\min_{x \in \mathbb{R}^p} \frac{1}{n} \sum_{i=1}^{n} \left(h(x, t_i) - y_i\right)^2 .$$

Tasks

1. Train the model with $d = 16$ using gradient method. Determine the best stepsize according to the criteria seen in class (adapt the code provided if needed).

2. Repeat for several values of d (this can be between 5 and 10 values) and propose a *scaling law* for the best stepsize as a function of d:

   $$\alpha(d) \approx \nu \, d^{\gamma}$$

   where you have to find numerically ν and γ. Illustrate your experimental choices with figures and discuss your results. **For this question, you do not need to provide an exhaustive discussion or figures on step-size tuning for every value of d. Focus on two cases (or three if really necessary). Likewise, you do not need to average results over multiple problem instances (e.g., different initializations or noise realizations), unless you wish to illustrate limitations. Please aim for a balance between rigor and concision.**

Evaluation (20 points total)

For each part:

- 1.5 points: Code quality, use of pytorch and autodiff: clean code, readable, well-commented; avoid raw copy-paste from LLMs.

- 1.5 points: Reproducibility and correctness of the implementation: the code has to work, others should be able to reproduce your results easily.

- 2 points: Presentation of results: nice figures, they can be inspired by those seen in class, balance between clarity and rigor/completeness.

- 2 points: Discussion of results, limitations, and flaws, with further numerical evidence if needed.

- 3 points: Experimental choices: follow benchmarking guidelines seen in class, explain all your choices and why some other choices did not work.